

Lecture 8

Constructors and Data Hiding

Spring 2025

Lecture's contents

- What is a constructor?
- Default Constructors (compiler-provided and user-provided)
- Parametrized Constructors
- Overloading Constructors with default arguments
- Data Hiding
- Public vs private member access specifier
- Accessor and mutator functions

Creating Objects using constructors

- Constructor is a special *class method* that is automatically called when a class creates its object
 - A constructor has same name class
 - Does not return a value, not even void
 - Initialize some or all member variables
 - Other actions possible as well
- Here are some of the examples of the constructors for the Time class, Date class, student class etc.

```
class Time {  
Public:  
Time() {}  
};  
int main() {  
Time T; }
```

← constructor

```
class Date {  
Public:  
Date() {}  
};  
int main() {  
Date D; }
```

← constructor

```
class student {  
Public:  
Student() {}  
};  
int main() {  
student S; }
```

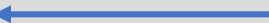
← constructor

Creating Objects using constructors

- C++ automatically *calls* a constructor for each object that is created, which helps ensure that objects are initialized properly before they're used in a program.

```
class DayofYear {  
Public:  
    int day;  
    string month;  
    int year;  
    DayofYear() {}  
  
    void display() {  
        cout<<day<<" "<<month<<" "<<year; }  
    }  
int main() {  
    DayofYear D;  
  
}
```

Constructor is called.



Default Constructors

- Constructor is a function in every class which is called when class creates its object
 - Basically it helps in initializing data members of the class
 - A class may have **multiple constructors**
- If a class does not *explicitly* include a constructor, the compiler provides a **default constructor**:
 - Default constructor is a constructor with no arguments, such as:

DayofYear() {}

- No data members initialized
- Do not rely on compiler-generated constructors

Class definition with default constructor

- Default Constructor is called when an object is declared without any arguments, such as:

```
class DayofYear {  
    public:  
    int day;  
    string month;  
    int year;  
  
    DayofYear () { //constructor explicitly provided by class  
        cout << "DayofYear's class object is created";  
    } // default constructor  
};  
  
int main () {  
    DayofYear D1;  
}
```

Program Output:

DayofYear's class object is created

Default constructor

- A default constructor is defined as a constructor with no arguments
- If a default constructor is provided by the user/compiler then:

```
DayofYear D1;    //That's Okay
```

```
DayofYear D2(); //Not okay.
```

- Because compiler sees a function declaration/prototype!
- If a default constructor is not defined then it is not possible to create an object as:

DayofYear D;
- There should always be a default constructor available in order to declare an object as above, whether user provided or compiler provided.

Constructor overloading

- A class may define more than one *constructor*, just like other functions
 - With different parameter lists
- Recall: a signature consists of:
 - Name of function
 - Parameter list
- Provide constructors for all possible argument-lists
- For our DayofYear class example the following are the default constructors and parameterized constructor:

```
DayofYear(); //default constructor
```

```
DayofYear(int d, string m, int y); //non-default or  
parameterized constructor
```


Constructor overloading: Date Class in C++

```
class DayofYear {  
public:  
    int day;  
    string month;  
    int year;  
  
    DayofYear() { cout<<... } //default constructor  
    DayofYear (int d, string m, int y ) //overloaded constructor  
    {  
        day = d;  
        month = m;  
        year = y;  
    }  
    void display();  
};  
  
int main() {  
    DayofYear today; //uses default constructor  
    DayofYear birthday(5,"January", 2000);  
}
```

Class exercise (cont'd from the previous)

- Create a simple class named 'Time'. The time class has three attributes : Seconds, minutes, hours,
- in addition it has the following method, display()→displays current time
- Also include:
 - A default constructor
 - And a parameterized constructor
- In main() create two objects of Time, named, noon and midnight.
- Using objects invoke the display method.

Constructor with default arguments

- Like other functions constructors can specify *default arguments*.
- The default arguments to the constructor ensure that, even if no values are provided in a constructor call, the constructor still initializes the data members.
- Example of constructors with default arguments are:

```
Time::Time(int hour=0, int min=0, int second =0);
```

```
Date::Date(int day=1,int month=1,int year = 2000);
```

- A constructor that defaults all its arguments is also a default constructor
 - That is a constructor that can be invoked with no arguments.
- There can be at most one default constructor per class.

DayofYear class containing constructor with default arguments

```
class DayofYear {  
public:  
    int day;  
    string month;  
    int year;  
  
    DayofYear (int d=1, string m="January", int y=2000 ) {  
        day = d;  
        month = m;  
        year = y  
    }  
    void display();  
};  
  
int main() {  
    DayofYear birthday; //uses default constructor  
    DayofYear today(20,"February",2025);  
}
```

Constructors are for constructing objects

- You may initialize all of your data members in class definition. However, you will still need a constructor to construct an object.
- In this case a default constructor is already provided by the compiler.

```
class DayofYear {  
public:  
    int day = 1;  
    string month = "January";  
    int year = 2000;  
  
};  
  
int main() {  
    DayofYear birthday;  
}
```

Constructors are for constructing objects

- What if you provide a customized constructor?
- If you write at least one constructor, the compiler will not generate a default constructor definition for your class.

```
class DayofYear {  
public:  
    int day = 1;  
    string month = "January";  
    int year = 2000;  
  
    DayofYear (int d, string m, int y ) {  
        day = d;  
        month = m;  
        year = y } };  
  
int main() {  
    DayofYear birthday; //Error: uses default constructor. But default  
    constructor is not given.  
    DayofYear today(20,"February",2025);  
}
```

Constructors are for constructing objects

- **Solution** for this is to write an empty default constructor

```
class DayofYear {  
public:  
    int day = 1;  
    string month = "January";  
    int year = 2000;  
    DayofYear() {}  
    DayofYear (int d, string m, int y ) {  
        day = d;  
        month = m;  
        year = y } };  
  
int main() {  
    DayofYear birthday; //No Error  
    DayofYear today(20, "February", 2025);  
}
```

Rectangle class containing constructor with default arguments

```
class Rectangle {  
public:  
    Rectangle(int w=5,int l=10){//constructor with default arguments  
        Width=w;  
        length=l;  
    }  
private:  
    int width;  
    int length;  
};  
  
int main() {  
  
    Rectangle  r1;  
    Rectangle  r2(2,40);  
    Rectangle  r3(3,60);  
  
}
```

```
Rectangle width = 5 and lenth = 10  
Rectangle width = 2 and lenth = 40  
Rectangle width = 3 and lenth = 60
```


Data Hiding

- *Data hiding* is an object-oriented programming (OOP) technique specifically used to hide internal object details (i.e., data members).
- *Data hiding* guarantees exclusive *data access to class members only* and protects and maintains object integrity by preventing intended or unintended changes and intrusions.

Data Hiding

- The following employee class violates the data hiding principle.

```
class employee {
Public:
    string name;
    int emp_id;
    int payscale;
    float salary;

employee(string n, int id, int scale, float wage) {
    //initialize all members here
}

};

int main() {
    employee emp("Muhammad", 57, 16, 50000);
    modify(emp);
}

void modify(Employee e) {
    e.salary = 100000;
    e.payscale = 17;
}
```

How to implement Data Hiding in C++?

- First, we ensure the data members of the class type are ***private*** (so that the user can not directly access them).
- The statements inside the bodies of member functions are already not directly accessible to users, so we don't need to do anything else there.
- Next, we ensure the member functions are ***public***, so that the user can call them.

Data Hiding using Private Member Access Specifier

- **private**

- Default access mode
- Data only accessible to member functions and **friends**(will be covered later)
- **private** members only accessible through the **public** class interface using **public** member functions.
- Declaring data members with access specifier private is known as *data hiding*.
- When a program creates (instantiates) an object, its data members are encapsulated (hidden) in the object and can be accessed only by member functions of the object's class.

- **public**

- Presents clients (or main() function) with a view of the services the class provides
- Data and member functions are accessible, i.e., Indicates that the data/function is “available to the public”—that is, it can be called by other functions in the program (such as main), and by member functions of other classes (if there are any).

Data Hiding

- The following employee class ensures data hiding principle.
- Data members declared private cannot be accessed directly.

```
class employee {  
private:  
    string name;  
    int emp_id;  
    int payscale;  
    float salary;  
public:  
    employee(string n, int id, int scale, float wage) {  
        //initialize all members here  
    }  
};  
  
int main() {  
    employee emp("Muhammad", 57, 16, 50000);  
    modify(emp);  
}  
  
void modify(Employee e) {  
    e.salary = 100000;  
    e.payscale = 17;  
}
```

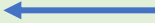
Data members
declared
private cannot
be accessed

Data Hiding

- The following employee class ensures data hiding principle.
- Data members declared private can be accessed by the class members.

```
class employee {  
private:  
    string name;  
    int emp_id, payscale;  
    float salary;  
public:  
    employee(string n, int id, int scale, float wage) {  
        //initialize all members here  
    }  
    void stepupscale() {payscale++; }  
    void increment() { salary = salary+50000; }  
};  
int main() {  
    employee emp("Muhammad", 57, 16, 50000);  
    emp.stepupscale();  
    emp.increment();  
}
```

Data members
declared
private can
Now be
accessed by
class members



Data Hiding

- Defining a function outside a class does not change it being **public** or **private**

```
class employee {  
private:  
    string name;  
    int emp_id, payscale;  
    float salary;  
  
public:  
    employee(string n, int id, int scale, float wage) {  
        //initialize all members here  
    }  
  
    void stepupscale();  
    void increment();  
};  
  
int main() {  
    employee emp("Muhammad", 57, 16, 50000);  
    emp.stepupscale();  
    emp.increment();  
}
```

```
void employee::stepupscale() {payscale++; }  
void employee::increment() { salary = salary+50000; }
```

Data members
declared private
can now be accessed
by class members

Rewriting dayofyear class

- Data members are now declared private
- Objects have no direct access

```
Class DayofYear {
Private:
    int day;
    string month;
    int year;

Public:
    DayofYear() {}//constructor declared as public
    void display () { //display can be accessed directly in the main
        cout<<" day: "<<day<<" month: "<<month<<" year: "<<year;
    };

int main(){
    Dayofyear D;
    D.display();
}
```

What if we want to modify a single data member of the DayofYear class
or
we want to access a single data member of the Dayofyear class??

Mutator and Accessor (Setter/Getter) Functions

- The two types of functions that each class should have:
 - Setter also called mutator
 - Getter also called accessor.
- These functions play a crucial role in data hiding by controlling access to private data members of a class.

Mutator and Accessor (Setter/Getter) Functions

- Every class has member functions that modify the data members, called **mutator** functions (**Setter**). For example, in our DayofYear example, we may have a setter function to set the date which is a private member of the class:

```
void setDate(int d);
```

- The setter function does not return anything and usually takes one or more arguments.
- The arguments passed are usually validated inside the setter function.

Mutator and Accessor (Setter/Getter) Functions

- Likewise, every class has member functions that only access and do not modify the data members; these methods are called **accessor** functions (**Getter**). For example, in our DayofYear example, we may have a getter function to get the year:

```
int getYear();
```

- The getter function does not take an argument and usually returns by value so a client can get the value saved in an object but cannot change the value. Later we will cover how to protect the getter function from accidentally changing the private data members.

Mutator and Accessor (Setter/Getter) Functions

```
class DayofYear {  
public:  
    DayofYear() { }           //default constructor  
    void setDate(int d){ day=d;}  
    int getYear() { return year; }  
    void input();  
    void output();  
private:  
    int day;  
    string month;  
    int year;  
};  
  
int main() {  
    DayofYear today;  
    today.setDate(28);  
    cout<<today.getYear();  
}
```